



GRIFFITH COLLEGE CORK
ASSIGNMENT TITLE SHEET
FACULTY OF COMPUTING SCIENCE

Course:	BSC
Stage/Year:	3
Module:	WT
Semester:	Semester 2
Assignment Number:	Assignment 2
Date of Title Issue:	16/03/26 (Rev2 29/03/26 to align with GCD)
Assignment Deadline:	14/04/26 by 11:59pm
Assignment Submission:	Upload to Moodle
Assignment Weighting:	30% of module

Assignment Title

WT - Assignment 2 - Server-side components

Artificial Intelligence Assessment Scale (AI AS):

This assignment is subject to AI permission option: (select all that apply)

No AI Use ☐
AI for Planning ☐
AI for Editing ☒
AI for Support Tasks ☐
AI for Collaboration ☐
Full Use of AI ☐

AI Assessment Scale Framework:

<https://www.griffith.ie/sites/default/files/2025-05/f-6.5-ai-in-learner-assessment-policy.pdf>

Please note:

- All AI generated content and process must be declared **and** screenshots of both prompts and responses must be included in an appendix.
- Any task that is not listed as permitted may be assumed to be prohibited.
- If it is determined that a learner has used an AI tool in any way that is not permitted by the chosen level, then they will be subject to the disciplinary procedures listed in the Academic Integrity and Misconduct policy regarding the misuse of AI in assignments.

Assignment Specific AI Instructions:

- AI may be used in **support tasks only** – eg. using the IDE'S AI assistant to generate in-place code suggestions.
- Generative AI may **not** be used to create blocks of code or whole sections of assignments.
- Where generative AI such as ChatGPT has been used for tutorial purposes, links must be provided to the chat along with screenshots of both prompts and responses included in an documentation appendix.
- **No AI may be used to write reports, documentation, or comments, and may not be used to create images, charts/graphs, wireframes, sitemaps, etc.**
- You may be called in for VIVA to explain your work if unauthorized/uncredited AI use is suspected.

Learning Outcomes

This assignment is assessing the following programme and related module learning outcomes:

LO4: Evaluate the architecture, System Analysis, design and features of server-side web application frameworks

LO5: Install and configure client and server-side web application frameworks and single-page applications

LO6: Plan, design and build web applications using both client-side and server-side web application frameworks.

Assessment Criteria

Assessment criteria applied to this assignment:

Assignment will be assessed on strength of ideas, clarity of writing, and quality of designs.

Submission Instructions and Plagiarism

Plagiarism:

All work must be your own. Any cases of suspected plagiarism will be thoroughly investigated and may be brought in for VIVA. All parties involved in cases of plagiarism will receive a 0 and their student record marked accordingly. Any submission that is found to be plagiarised will receive a grade of 0 without the possibility of resubmission.

By submitting your assignment you accept that you understand what plagiarism is and that your assignment is not plagiarised in any way.

You may not submit work or partial work that has previously been submitted for assessment.

Instructions for submission:

Upload your report as a **PDF** to Moodle. Each team submits **only one copy**.

Submission naming convention: Save your proposal and UI design as a PDF file with the name of your project (only PDF is accepted). Be sure include all members' names in the coversheet.

Penalties

Late Penalties:

Standard penalties will be applied to work that is submitted late, as per faculty guidelines. <https://moodle.griffith.ie/mod/resource/view.php?id=18202>

Penalties that apply to this assignment:

-100 marks - No demo **(no points will be awarded if you do not demo your work)**

-10 marks - Your team did not submit all files necessary for your project to be functioning

-10 marks - Your team did not submit a URL to your app hosted online.

Project Overview

In this assignment, you will work in your project groups to implement server-side web component(s) written in Node and Express JS to perform the functionality of your project.

You are encouraged to add enough service and behaviour to your application to fulfil the vision that you set out in your project proposal. Please consider the following as minimum requirements only - you are encouraged to get creative and do more than this (to optimise your marks), but this is what will be required to pass.

The overall behaviour of your app must be a single unit. It must not be disconnected. All parts / components must be properly connected. That is, once a user enters your app through a web browser, there is an appropriate navigation system and / or an execution flow allowing a user to achieve his / her goal(s). No styling need be present on the front-end

Overall, your system must be functioning. Your entire project must work as expected; it must provide normal behaviour, given the domain application of the project.

Project requirements:

These requirements must be implemented in **Node and Express:**

- Users Management
- CRUD
- Validation - server side
- Coding Standards
- Deploy Online

Suggested features / aspects you may make use of to improve the functionality of your project and fulfil requirements:

Users Management

- Support multiple users
- Returning users must be able to access their existing data, records, or previous states; for example, existing orders, previously stored data, last request, recent setting / configuration / preference.
- Users must be able to retrieve and view data previously stored in a database and use them in some ways to serve some purposes
- Users must be able to add, update and / or delete (in some ways, you decide) data to the database, i.e., data being stored must reflect the user inputs and / or the application states.
- You decide the business logic, whether users can modify and / or delete other users' data or levels of permissions may be incorporated in your project.
- SESSION - state maintenance mechanism
- COOKIE - stores information on the client

CRUD

- Create (POST): Adds new data.
- Read (GET): Retrieves data.
- Update (PUT/PATCH): Modifies existing data.
- Delete (DELETE): Removes data.

Please note: **a database is required.**

Validation - server side

- Implement form submission and handling
- Validate form data
- Provide user-appropriate error messages
- If no error message is provided, other reasonable and appropriate handling (such as auto-complete or auto-correction with defaults) and / or some forms of user-appropriate feedback are acceptable.
- Perform server-side input validation

Coding Standards

- Use good coding style
- Make identifier names understandable

- Use proper and consistent indentation
- Use comments. All code must be commented

Deploy Online

- Deploy your system online with <https://render.com>.
- Submit a URL to your app hosted online along with your source code. Use this URL to demo your app.
- IMPORTANT: The URL is not for a “website” in this stage — it is for the API to have something to contact and CRUD management of database entities. There does not need to be any client-facing React components. If your API responds correctly to requests from a template-driven Express CRUD app, then your app is working.

Required outputs:

What to include

- All of your project files. In correct folder structure as discussed in class. The files submitted should be **all** the files necessary for your project to run on your server, or on a similarly configured server.
- Your database
- A readme.md file detailing:
 - o your work so far and the contributions of each team member to this iteration of the project.
 - o Include all members' names as authors of the programs/files based on contribution.
 - o References to any tutorials, guides, third-party resources etc.
- A coversheet with all contributing students' full names and student numbers.

How to submit

- All of the above should be in the root folder of your site. Save your site as a **ZIP file with the following name <groupname><assignment2><AppName>.zip** (only ZIP is accepted) **No RAR files!**
- Be sure include all members' names in the coversheet.
- Each team submits only **one** copy.
- Your site information readme.md in your site root folder
- cover sheet as a PDF in your site root folder
- Upload your site as a ZIP file to **Moodle**

Division of Labour

All team members are expected to contribute equally to the project. At each iteration the team will fill state how the work was divided amongst the team. E.g. If the work was split evenly per team member, then that is fine and simply say “Work was evenly divided”. If this was not the case, then state with a summary sentence or percentage.

Grading

	Components	Total Marks
Correctly and properly implement:	- Correctly and properly implementation of functions declared in Assignment 1 - Where changes have occurred from your initial plan, be careful to state those clearly in the readme. - These implementations should all be listed in the readme file.	10
Proper implementation of form submission and handling & validation	- Appropriate server-side input validation - Properly validate form data - Provide user-appropriate error messages. If no error message is provided, other reasonable and appropriate handling (such as auto-complete or auto-correction with defaults) and / or some forms of user-appropriate feedback are acceptable.	10
Sessions	Correctly implement and properly support multiple users, using the server-side sessions in Express	5
Cookies	Correctly implement and properly maintain state of the application using cookies	5
Database	Correctly implement and properly support multiple sessions, using a database	10
Users / CRUD	Although you are implementing the back end you must setup the back end so that: - Users must be able to retrieve and view data previously stored in a database and use them in some ways to serve some purposes. - Users must be able to add data to the database, i.e., data being stored must reflect the user inputs and / or the application states. - Users must be able to update and / or delete data stored in the database, i.e., data being stored must reflect the user inputs and / or the application states. <i>This will be done through the interface of your system, not direct access to the database.</i>	20
Coding style	- Use good coding style: readability, style, and maintainability - Make identifier names understandable - Use proper and consistent formatting - Use clear comments describing intended functionality	10
User demonstration	Program is demonstrated clearly Code segments clearly explained	10
App hosted online	Your application must be hosted online	10
Documentation	You clearly describe your implementations in your readme document	10

	• All work is correctly cited • All work presented professionally	
Total		100

Extra	You have researched and implement functionality beyond what we covered in class and the requirements of this brief. Mark this very clearly in your comments!	10
-------	--	----